

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1508

COURSE OUTCOME COVERED

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I, **G. Umesh Chandra Yadav/ 192525424** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *G. Umesh*

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.
2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

Application Based Questions Rubric

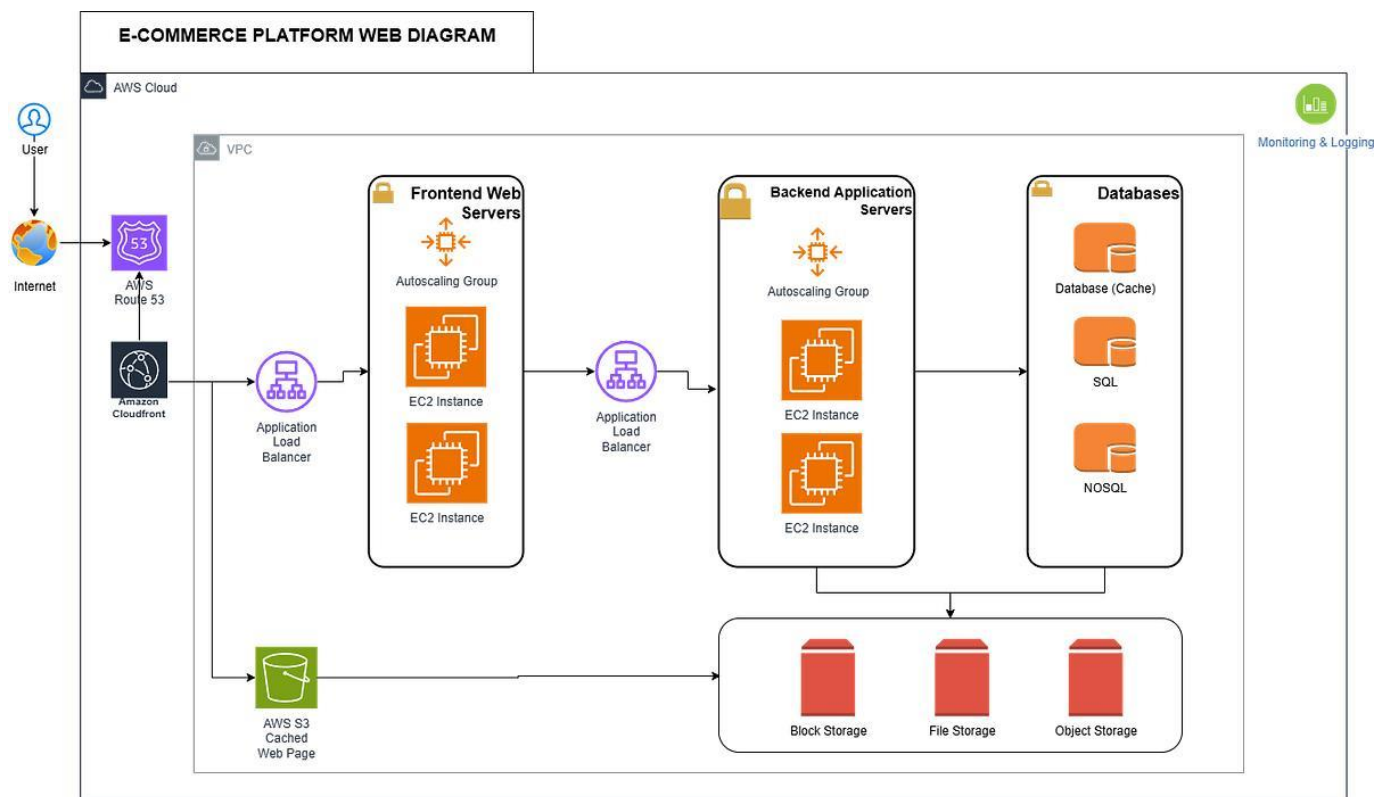
Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

Application Based Questions – Detailed Solutions

1. Auto-Scaling and Load Balancing for Seasonal E-Commerce Traffic

Scenario:

A company operates an e-commerce website where traffic increases suddenly during seasonal sales such as **Diwali, Black Friday, Christmas, or New Year sales**. Thousands or millions of customers may access the website simultaneously. If the infrastructure cannot handle the sudden increase, the website may become slow or unavailable.



How Auto-Scaling Handles Traffic

Cloud auto-scaling automatically increases or decreases computing resources according to workload demand.

For example:

- During normal periods, the company may run **3 application servers**.

- When traffic increases significantly, auto-scaling can automatically increase this to **10 or 20 servers**.
- When the sale ends and traffic decreases, unnecessary servers are removed.
- The company therefore pays mainly for the resources it actually needs.

A typical architecture is:

Users → Load Balancer → Auto-Scaled Application Servers → Database/Cloud Storage

Role of Load Balancing

A **load balancer** distributes incoming customer requests across multiple application servers.

For example, if 10,000 customers are accessing an online shopping website:

- Server 1 handles some requests.
- Server 2 handles another group.
- Server 3 handles another group.
- Additional servers are added automatically when required.

This prevents a single server from becoming overloaded.

How It Reduces Downtime

Auto-scaling and load balancing work together to improve availability.

1. **Traffic increases** → monitoring detects high CPU/request levels.
2. **Auto-scaling adds servers.**
3. **Load balancer distributes traffic** among the available servers.
4. If one server fails, the load balancer can stop sending traffic to it.
5. Other healthy servers continue serving customers.
6. When traffic decreases, auto-scaling removes unnecessary instances.

Therefore, the failure or overload of one server does not necessarily make the entire application unavailable.

Improvement in User Experience

This approach provides:

- Faster page loading

- Reduced response time
- Fewer failed requests
- Better checkout performance
- Higher availability
- Ability to handle sudden traffic spikes
- Better scalability during peak sales

For example, during a major **e-commerce flash sale**, customers may simultaneously search products, add products to carts, make payments, and check order status. Auto-scaling provides additional compute capacity while the load balancer distributes requests efficiently.

Role of Monitoring Tools

Monitoring tools continuously observe:

- CPU utilization
- Memory usage
- Network traffic
- Number of requests
- Response time
- Error rate
- Server health
- Application performance

For example:

CPU > 80% → Monitoring detects overload → Auto-scaling launches additional instances → Load balancer distributes traffic

Monitoring also helps administrators identify problems before they cause major downtime.

Real-World E-Commerce Example:

Consider an online retailer running a **Diwali sale**.

Normal traffic may require 5 application servers. When a flash sale begins, traffic suddenly increases by 10 times. The monitoring system detects increasing CPU utilization and request

rates. Auto-scaling launches additional application instances, while the load balancer distributes customers among the servers.

After the sale, traffic returns to normal, so the cloud platform automatically reduces the number of instances.

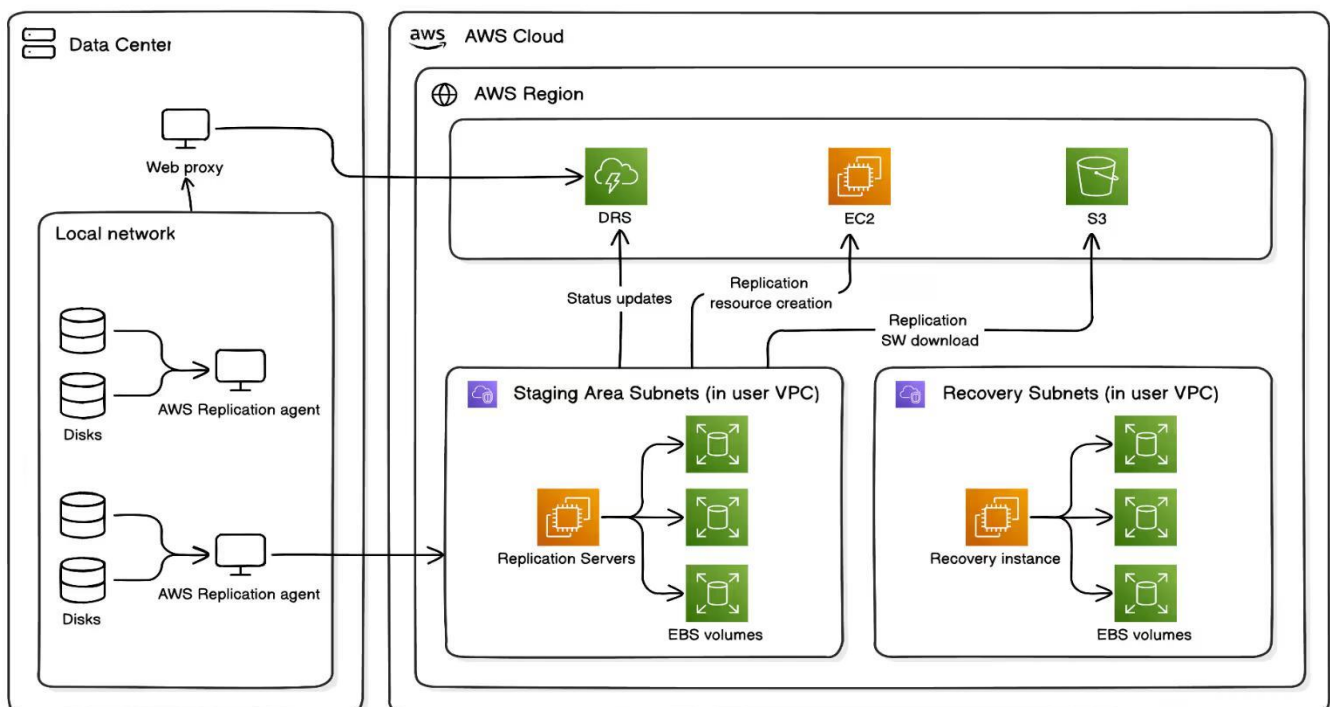
Conclusion:

Auto-scaling provides elasticity, while **load balancing provides efficient traffic distribution and availability**. Monitoring tools connect these mechanisms by continuously measuring system health and triggering scaling actions. Together, they reduce downtime, maintain application performance, and provide a better shopping experience while controlling infrastructure costs.

2. Secure Cloud Storage for Large-Scale Backup Data

Scenario:

An organization generates a large amount of important data that must be backed up securely. Traditional storage systems require organizations to purchase and maintain physical storage hardware. Cloud storage provides scalable and highly durable storage without requiring the company to maintain all the physical infrastructure itself.



How Cloud Storage Provides Durability?

Durability refers to the ability of a storage system to protect data from accidental loss or hardware failure.

Cloud storage providers achieve high durability using techniques such as:

- Data replication
- Redundant storage systems
- Multiple storage devices
- Multiple availability zones
- Geographic replication
- Automated integrity checking

For example, instead of storing one backup copy on one disk, multiple copies can be maintained across independent infrastructure.

Data Redundancy:

Redundancy means keeping additional copies of data so that failure of one component does not destroy the only copy.

Example:

Primary Backup → Copy 1 → Copy 2 → Disaster Recovery Copy

If one storage device fails, another copy can be used.

For critical organizational data, copies may be stored in different geographical locations so that even a regional disaster does not destroy every backup.

Encryption for Backup Protection

Backup data may contain sensitive information such as:

- Customer records
- Employee information
- Financial records

- Medical information
- Business documents
- Database backups

Encryption protects this information by converting readable data into an unreadable form.

Two important forms are:

Encryption at rest:

Protects data while it is stored in cloud storage.

Encryption in transit:

Protects data while it moves between the organization and cloud storage.

Thus, even if unauthorized individuals obtain encrypted storage data, they cannot easily understand the content without the required cryptographic keys.

Access Control

Encryption alone is not sufficient. The organization should also implement **Identity and Access Management (IAM)**.

Access control can ensure that:

- Only authorized administrators access backups.
- Users receive only required permissions.
- Backup deletion permissions are restricted.
- Administrative actions are logged.
- Multi-factor authentication can be required.

For example:

- **Backup Operator → Read/Write Backup**
- **Auditor → Read Only**
- **Regular Employee → No Backup Access**

This follows the **principle of least privilege**.

Cost Advantages

Cloud backup can be more economical than traditional storage because organizations do not necessarily need to:

- Purchase large storage arrays in advance
- Maintain physical servers
- Replace failed disks manually
- Operate dedicated storage rooms
- Pay for unused storage capacity
- Maintain extensive backup infrastructure

Cloud storage can scale according to requirements.

For example:

Traditional approach: Purchase 100 TB storage even though only 40 TB is initially needed.

Cloud approach: Start with the required capacity and expand as data grows.

Cloud providers also offer different storage tiers, allowing older and less frequently accessed backups to be moved to lower-cost storage.

Disaster Recovery Example

Suppose a company's primary datacenter is affected by a **fire or major hardware failure**.

The company can:

1. Maintain backups in cloud storage.
2. Replicate critical data to another geographical region.
3. Recover databases and application servers in a secondary environment.
4. Restore the latest valid backup.
5. Resume business operations.

For example:

Primary Datacenter Failure → Cloud Backup → Secondary Region → Restore Database → Restart Application

This reduces data loss and business interruption.

Conclusion:

Cloud storage provides durability through redundancy and replication, while encryption and access control protect confidentiality. Scalable storage and pay-as-you-use models can reduce infrastructure costs. Cloud backups are especially valuable for disaster recovery because organizations can restore important data even when their primary infrastructure becomes unavailable.

3. CDN and Global Deployment of a Mobile Application

Scenario

A mobile application company wants users from different countries to access its application quickly. If all application content is delivered from a single server located in one country, users who are geographically far away may experience increased latency.

A **Content Delivery Network (CDN)** helps solve this problem.



What Is a CDN?

A **Content Delivery Network** is a geographically distributed network of servers called **edge servers or edge locations**.

Instead of every user requesting content from the main origin server, frequently requested content can be served from a nearby edge location.

Example:

Origin Server → CDN Edge Locations → Users

How CDN Reduces Latency?

Suppose the main application server is located in the United States.

A user in India would have to send a request over a long network path to the US and wait for the response.

With a CDN:

Indian User → Nearby CDN Edge → Content

The content travels a shorter distance, reducing latency.

What Can Be Delivered Through a CDN?

CDNs are particularly effective for:

- Images
- Videos
- JavaScript files
- CSS
- Application assets
- Game updates
- Software downloads
- Streaming content

Role of Edge Servers

Edge servers are located closer to users geographically.

For example:

User Location	Possible Nearby Edge Location
India	Mumbai/Chennai/Delhi region
Europe	Frankfurt/London region
USA	Virginia/California region
Asia-Pacific	Singapore/Tokyo region

When cached content is available at an edge location, users do not always need to communicate directly with the origin server.

Improvement in User Experience

CDNs can provide:

- Lower latency
- Faster content delivery
- Faster application loading
- Reduced origin server workload
- Better streaming performance
- Improved scalability
- More consistent performance across regions

This is particularly important for applications where users expect near-real-time responses.

Cloud Support for Global Availability

Major cloud providers offer global infrastructure consisting of:

- Multiple geographic regions

- Availability zones
- CDN/edge networks
- Global load balancing
- Distributed storage
- Managed databases
- Disaster recovery capabilities

An application can therefore deploy services in multiple regions rather than relying on one physical datacenter.

Real-Time Streaming Example

Consider a global sports streaming application.

During a major cricket match, millions of users may simultaneously watch live video.

A single origin server could become overloaded.

A CDN can distribute video content through edge locations:

Streaming Origin → CDN → Edge Servers → Millions of Viewers

A viewer in India receives content from a nearby edge location, while a viewer in Europe receives it from a European edge location.

This reduces the distance that content must travel and helps the system serve a large global audience.

Gaming Example:

For an online multiplayer game, CDN infrastructure can distribute:

- Game updates
- Installation files
- Maps
- Textures
- Static game assets

Regional cloud infrastructure can also place application services closer to players, reducing network latency.

Conclusion:

CDNs improve global application performance by delivering content from geographically distributed edge servers. Cloud providers complement CDN services with multiple regions, availability zones, global networking, and distributed services. This allows mobile applications, streaming platforms, and games to provide faster and more reliable experiences to users worldwide.

4. Cloud Compliance and Security for a Financial Institution

Scenario

A financial institution handles highly sensitive information such as:

- Bank account details
- Credit/debit card information
- Customer identities
- Transaction records
- Loan information
- Payment information

Therefore, cloud deployment must provide strong security, auditing, compliance, and control over where data is stored.

Regulatory Compliance

Cloud providers offer services and controls that help organizations meet various regulatory and security requirements.

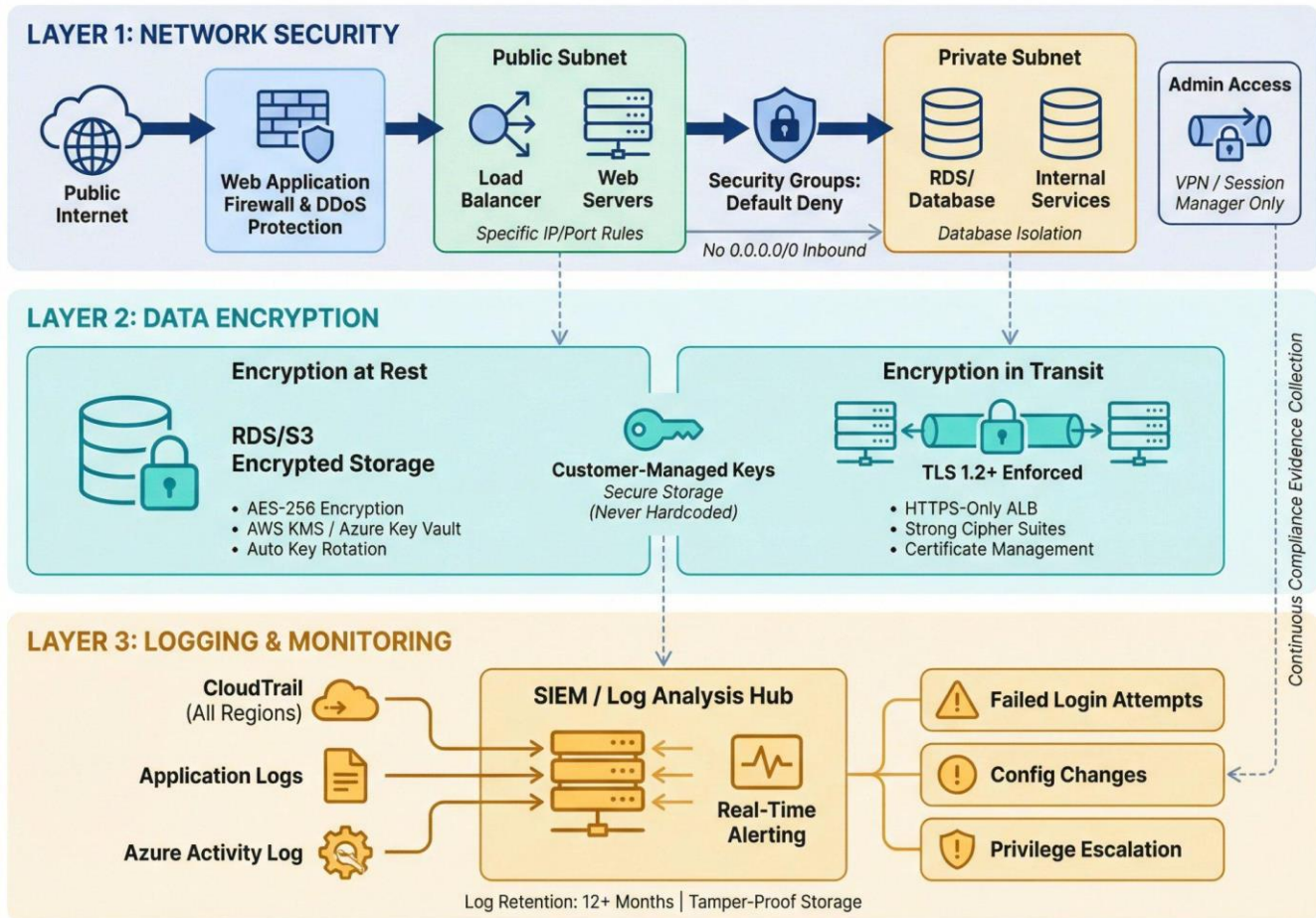
Depending on the organization's location and business activities, relevant requirements may include:

- Financial-sector regulations
- Data protection requirements
- Privacy regulations
- Industry security standards

- Internal organizational policies

However, using a compliant cloud provider **does not automatically make the customer compliant**. The organization must configure its cloud environment correctly and follow the applicable regulations.

MODERN CLOUD SECURITY ARCHITECTURE: 3-LAYER DEFENSE-IN-DEPTH FOR COMPLIANCE



Auditing

Auditing provides evidence of:

- Who accessed data
- When the access occurred
- What operation was performed
- Which system was accessed
- Whether configuration changes occurred

- Whether suspicious activity occurred

For example:

Employee A → Accessed Transaction Database → 10:30 AM → Read Transaction Record

Such information can be reviewed during security investigations and compliance audits.

Role of Logging:

Cloud logging records important activities such as:

- User logins
- Failed login attempts
- Database access
- API calls
- Configuration changes
- File access
- Administrative activities

Logs can help identify unauthorized activity.

For example:

Multiple Failed Logins → Security Monitoring Detects Abnormal Activity → Account Restricted → Security Team Investigates

Role of Monitoring:

Monitoring continuously observes the security and performance of cloud infrastructure.

It can detect:

- Unusual login behaviour
- Unexpected network traffic
- Excessive API requests
- Abnormal database activity
- Resource changes

- Security events

Monitoring combined with alerts allows the organization to respond quickly.

Role of Encryption

Financial data should be protected using encryption.

Encryption at rest:

Protects stored databases, files, backups, and storage volumes.

Encryption in transit:

Protects information while moving between users, applications, databases, and cloud services.

For example:

Customer → Encrypted Connection → Banking Application → Encrypted Database

Encryption reduces the risk of sensitive information being exposed.

Access Control

The financial organization should implement strong IAM controls.

Examples:

- Role-based access control
- Multi-factor authentication
- Least-privilege permissions
- Privileged access management
- Separation of duties

For example:

Customer Service Employee: Can view permitted customer information.

Database Administrator: Can maintain database infrastructure.

Financial Auditor: Can access audit records but cannot modify transactions.

This reduces the possibility of unauthorized modification.

Data Sovereignty

Data sovereignty refers to requirements concerning the country or jurisdiction in which data is stored and processed.

If financial regulations require customer information to remain within a particular country, the organization can:

1. Select an appropriate cloud region.
2. Store sensitive databases in that region.
3. Restrict replication to approved regions.
4. Control cross-border data transfers.
5. Monitor data movement.
6. Maintain appropriate audit records.

Sensitive Financial Transaction Example

Consider an online banking application.

A customer transfers ₹50,000 to another account.

The system should provide:

Customer Authentication → Transaction Request → Authorization → Encrypted Processing → Database Update → Audit Log

The audit system can record:

- Customer/account identifier
- Transaction timestamp
- Transaction amount
- Application/service involved
- Security events
- Administrative changes

Sensitive transaction data should be encrypted, while access should be restricted to authorized systems and personnel.

Conclusion:

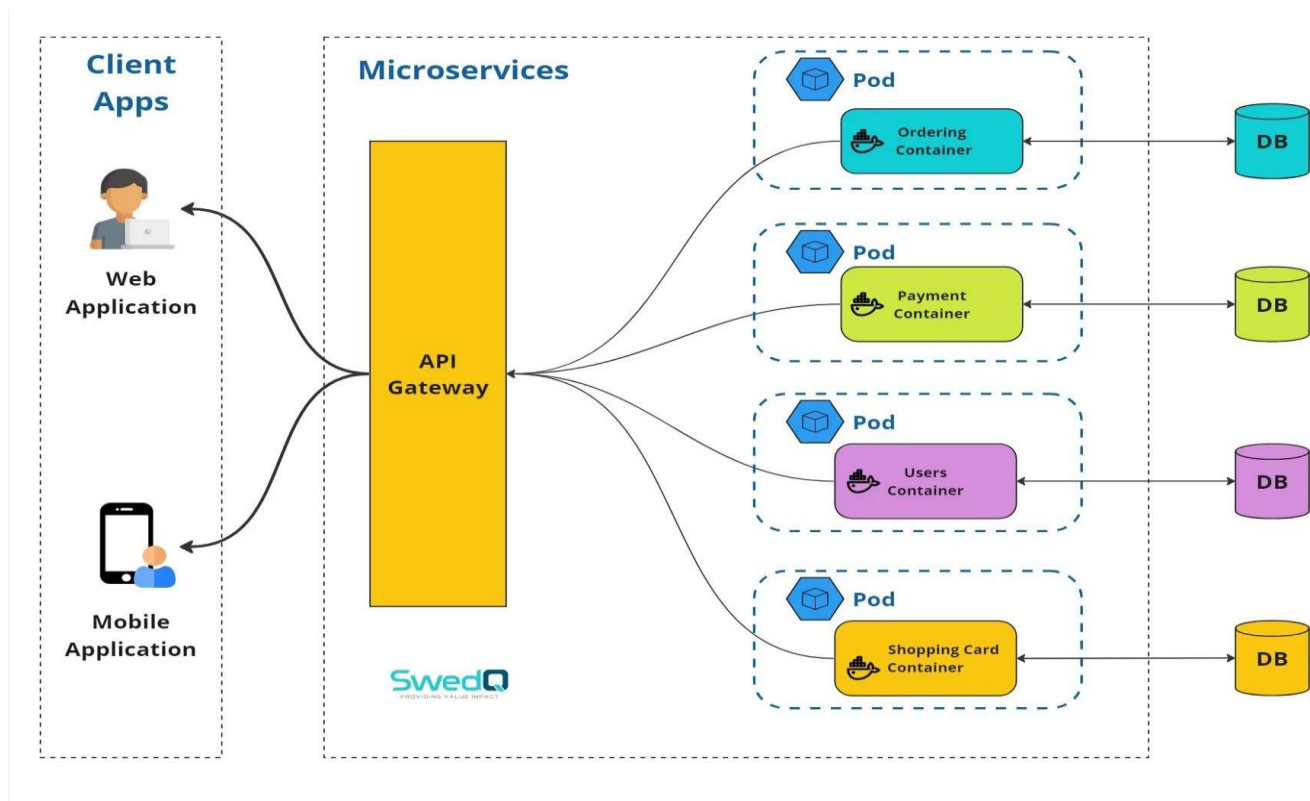
For financial institutions, cloud security requires multiple layers rather than relying on one mechanism. Encryption protects data, IAM controls access, logging provides accountability, monitoring detects suspicious activity, and compliance controls support regulatory requirements. Data sovereignty can be addressed through appropriate region selection, data-location controls, and restrictions on cross-border processing.

5. Containerization, Kubernetes, and Microservices

Scenario:

A development team wants to deploy an application efficiently across different cloud environments. Traditional application deployment can be difficult because the application may depend on a specific operating system, library, runtime, or configuration.

Containers solve many of these problems by packaging an application together with its required dependencies.



Containers vs Virtual Machines

Both containers and virtual machines provide isolation, but they work differently.

Virtual Machines:

A VM includes:

Physical Hardware → Hypervisor → Guest OS → Libraries → Application

- A VM runs a complete guest operating system on virtualized hardware.
- Each VM has its own OS, libraries, and applications.
- VMs generally require more CPU, memory, and storage.
- They take more time to start compared with containers.
- VMs provide strong isolation between workloads.
- Suitable for running different operating systems on the same physical server.

Containers:

Containers share the host operating system kernel while isolating applications.

Architecture:

Physical Hardware → Host OS → Container Runtime → Containers → Applications

- Containers package an application and its dependencies together.
- Containers share the host operating system kernel.
- They require less memory and storage than VMs.
- Containers start very quickly.
- More containers can run on the same hardware compared with VMs.
- They are highly suitable for microservices and cloud-native applications.
- Containers are easier to move between different cloud environments.

Comparison:

Feature	Virtual Machine	Container
Operating system	Separate guest OS	Shares host kernel
Resource usage	Higher	Lower
Startup	Usually slower	Usually faster
Isolation	Strong VM-level isolation	Process/application isolation
Density	Fewer workloads per host	More workloads per host
Portability	Good	Very high
Typical usage	Full OS workloads	Microservices and application deployment

Performance and Resource Usage

Containers are generally lightweight because they do not require a separate complete guest operating system for every application.

For example:

VM approach:

10 applications → potentially 10 guest operating systems.

Container approach:

10 applications → containers share the underlying operating system kernel.

This can improve resource utilization and allow more application workloads to run on the same infrastructure.

Role of Kubernetes

When an organization has only a few containers, manually managing them may be possible. But a large application may have hundreds or thousands of containers.

Kubernetes is a container orchestration platform that helps manage containerized workloads.

It can automate:

- Deployment
- Scaling
- Load balancing
- Service discovery
- Container scheduling
- Health checks
- Rolling updates
- Self-healing
- Restarting failed containers

How Kubernetes Simplifies Deployment

Suppose an application requires:

3 frontend containers + 5 payment containers + 4 database-service containers

Instead of manually managing every container, Kubernetes can manage these workloads as services.

If a container fails:

Container Failure → Kubernetes Detects Failure → Replacement Container Started

If traffic increases:

High Traffic → Kubernetes Scaling → Additional Containers

This improves application availability and reduces manual administration.

Portability Across Cloud Environments

Containers provide a consistent packaging format.

For example, an application container can be deployed on:

- Private cloud
- Public cloud

- Hybrid cloud
- On-premises infrastructure
- Different Kubernetes environments

The application can therefore move between cloud environments with fewer changes than an application tightly coupled to a particular server environment.

Microservices Example

Consider an online shopping application.

Instead of building the entire application as one large application, it can be divided into microservices:

E-Commerce Application

- User Service
- Product Service
- Shopping Cart Service
- Order Service
- Payment Service
- Notification Service
- Inventory Service

Each service can run in its own container.

For example:

- User Service → Container 1
- Product Service → Container 2
- Cart Service → Container 3
- Payment Service → Container 4
- Order Service → Container 5

Kubernetes can manage these containers and scale individual services independently.

For example, during a major sale:

Product Service Traffic ↑ → Scale Product Service

while the less-used services may remain at their normal capacity.

Advantages of This Approach

Containerization provides:

- Lightweight deployment
- Faster startup
- Better resource utilization
- Application isolation
- Consistent environments
- Easier testing
- Easier deployment
- Portability
- Independent microservice scaling

Kubernetes further provides automated orchestration, scaling, service discovery, and recovery.

Conclusion

Containers are more lightweight than traditional VMs because they share the host operating system kernel. Kubernetes simplifies management of large numbers of containers by automating deployment, scaling, service discovery, load balancing, and recovery. This combination is particularly suitable for microservices-based applications that need to scale independently and operate across different cloud environments.
